DOCUMENT NAME

Document title	Project Plan		
Author(s)	Dung, Mai Van		
Role	Team Members		
Date	10/12/2025	**File Name:**	C1SE.40_ProjectPlan_v2.0.docx

REVISION HISTORY

Version	Date	Comments	Author	Approval
1.0	29/08/2025	Initial release	All members	
1.2	16/10/2025	Updated	All members	
1.3	15/11/2025	Updated	All members	
1.4	05/12/2025	Updated	All members	
2.0	10/12/2025	Official	All members	

TABLE OF CONTENTS

LIST OF TABLE

1. Project Overview

1.1. Project Description

Table 1. *Project Description*

Project code	C1SE.40	**Customer**	N/A
Project Type	External	**Application type**	Open Resources
Project Category	Development	**Scrum master**	Thang, Huynh Minh

1.2. Scope and Purpose

1.2.1. Purpose

The purpose of the Elderly Health Monitoring and Alert System is to develop a reliable and user-friendly platform that continuously monitors the health status and daily activities of elderly individuals, ensuring their safety and well-being.

The system aims to provide real-time health tracking, automatic alert notifications, and remote caregiver access to essential data such as heart rate, movement activity, and daily routines. In the event of irregularities or potential health risks (e.g., inactivity, abnormal vital signs, or a detected fall), the system will immediately send notifications to caregivers or family members through mobile or web interfaces.

The primary goal of the system is to enhance elder safety, independence, and communication between elders and caregivers through reliable health data monitoring and timely alert mechanisms.

1.2.2. Scope

The project will focus on the design and development of a smart fall detection system with the following boundaries:

- Core Features:
    + Continuous health monitoring (e.g., activity tracking, vital sign collection via sensors or devices).
    + Real-time alerts for abnormal health readings or inactivity detection.
    + Secure data storage and visualization dashboards for caregivers and healthcare professionals.
    + Mobile and web-based interfaces for secure access and caregiver management. Integration with IoT devices and cloud-based services for remote monitoring.
- Inclusions:

    - System architecture design including sensors, communication modules, and databases.
    - Development of fall detection logic and monitoring rules to identify potential health anomalies or fall risk.
    - Development of monitoring and alert management interfaces for caregivers and family members.
    - Testing under simulated health conditions to ensure reliability and accuracy.
- Exclusions
    - The system does not perform full medical diagnosis or treatment recommendations.
- It does not replace professional healthcare services or emergency response unit
- Target Users
    - Elderly individuals living independently or under family supervision.
    - Caregivers and family members responsible for elder well-being.
    - Healthcare professionals who require remote monitoring data for evaluation.

This scope ensures the project remains focused on monitoring, alerting, and caregiver communication, while allowing for future expansion into AI-based analytics and broader healthcare integration.

1.3. Assumptions and Constraints

Table 2. *Assumptions and Constraints*

No	Description	Note
Assumptions		
1	Stakeholders may request changes or updates to project documentation during development	External Interfaces
2	The project may not be able to fully address every requirement due to technical or resource limitations	Quality
3	The overall project timeline can be adjusted (extended or shortened) depending on stakeholder needs or resource availability	Schedule
4	New features or requirements may arise during the project development life cycle	Scope
Constraints		
1	The project must satisfy all functional requirements defined in the User Stories	Schedule
2	The system must comply with applicable data privacy and security regulations	Security

1.4. Project Objectives

1.4.1. Standard Objectives

Table 3. *Standard Objectives*

Metrics	Unit	Committed	Note
Start Date	dd-mm-yyyy	17/08/2025	
End Date	dd-mm-yyyy	12/12/2025	
Team Size	Person	5	
Duration: Number of workdays	Days	94	
Number of work hours per day for one engineer	Person-hour	6	

1.4.2 Specific Objectives

- Functional Objectives
    + Real-time Health Monitoring: Continuously collect and display health data (e.g., movement, activity level, heart rate) from elderly users through sensors or wearable devices.
    + Fall Detection and Risk Alert: Automatically detect sudden falls, inactivity, or unusual readings using integrated sensors and generate immediate, priority alerts.

  + Alert and Notification System: Automatically send timely alerts to care gives or family members via mobile notifications, email, Telegram Bot, or SMS upon detection or abnormal conditions potential risks
  + Caregiver Dashboard: Provide an intuitive dashboard that visualizes health data, alert history, and user activity trends for caregivers and healthcare professionals.
  + User Profile Management: Enable secure user account creation, authentication, profile updates, and personalized monitoring configurations for each elder and caregiver.
  + Data Logging and Analytics: Store historical health data securely for long-term tracking and AI-assisted analysis to improve prevention and response accuracy.
- Quality Goals
  + High Reliability: Ensure continuous operation with minimal downtime and high data transmission accuracy.
  + Security and Privacy: Protect sensitive health and personal information through secure authentication, encryption, and compliance with data protection standards.
  + Scalability: Design the system to handle multiple users and devices simultaneously without performance degradation.
  + Good User Experience: Provide a simple, clear, and accessible interface suitable for elderly users and non-technical caregivers.
  + Maintainability: Develop a modular and well-documented system to support future upgrades and integration with additional health devices or AI modules.
  + NFR1 – Performance (Smooth video processing)
  + NFR2 – Reliability (Achieves 95% uptime).
  + NFR3 – Security (Implements encryption, secure login).
  + NFR4 – Usability (Intuitive dashboard for non-technical users).
  + NFR5 – Scalability (Architecture supports future enhancements).
  + NFR6 – Privacy Compliance (Stores only event metadata and short clips).

1.5. Critical Dependencies

At this stage, the project has no identified critical dependencies. All tasks are planned to be carried out within the project team's capacity without reliance on external deliverables or third-party schedules.

1.6. Project Risk

Table 4. *Project risk*

Risk	Description	P	I	Risk Exposure	Mitigation Strategy
Lack of technical expertise.	The project team may lack the required technical skills or experience, leading to delays or suboptimal outcomes.	4	4	16	Provide training for team members and seek external consultation if necessary.
Development delays	Limited experience or unexpected challenges may cause the development timeline to extend beyond the plan.	5	4	20	Create a phased development plan and release content incrementally to maintain progress.
Data security risks	Data breaches, unauthorized access, or system hacks may result in data loss or legal issues.	4	5	20	Implement robust security protocols, regular audits, and compliance with data protection regulations.
Poor user adoption	A complicated or confusing user interface may discourage users from adopting the system.	4	4	16	Simplify the user interface, conduct usability testing, and provide tutorials and customer support
Team conflicts	Lack of communication skills or unclear communication channels may cause misunderstandings or disagreements within the team.	4	4	16	Establish clear communication guidelines, regular meetings, and a collaborative problem-solving environment.

2. Project Development Approach

2.1. Technical Process

We use Agile methodology in our project, specifically Scrum, in addition to using software tools to manage work, assign tasks to team members such as Zalo, Messenger, Trello, Google Drive, Discord and source code management tools such as git. And to meet the project MySQL requirements, we use JavaScript and ReactJS for web development, Nodejs and React Native for application development with MySQL database.

2.1.1 Reasons for selecting

Because we can be flexible with our changing requirements. During the project development, it helps our project run smoothly without any interruption.

2.1.2 Agile Methodology

- Advantages of Agile:
  - + Adaptability: Easy to accommodate changes in requirements.
  - + Customer involvement: The customer gets regular updates and can provide feedback during each sprint.
  - + Faster delivery: Working software is delivered in short iterations.
- Agile Methodology
  - + Agile is a software development methodology focused on iterative and incremental delivery. The Agile Manifesto emphasizes four key values:
    - o Individuals and interactions over processes and tools.
    - o Working software over comprehensive documentation.
    - o Customer collaboration over contract negotiation.
    - o Responding to change over following a plan.
- Agile Project Life Cycle:
  - + Concept: Initial planning and requirement gathering.
  - + Inception: Building the project plan, creating user stories.
  - + Iteration/Construction: Development sprints, testing, and feedback cycles.
  - + Release: Final testing, deployment, and product launch.
  - + Maintenance: Post-launch updates, bug fixes, and support.
- Scrum Process

Scrum is a popular Agile framework, based on iterative development cycles called sprints.

+ Principles:

Collaboration, iterative progress, and team accountability.

+ Steps:

- Product Backlog: The complete list of tasks.
- Sprint Planning: Select tasks for a sprint (usually 2-4 weeks).
- Daily Scrum: Short daily standup meetings to discuss progress.
- Sprint Review: Review completed work at the end of the sprint.
- Sprint Retrospective: Analyze what worked well and what didn't.

+ Roles in Scrum:

- Product Owner: Defines product features and manages the backlog.
- Scrum Master: Facilitates the Scrum process and removes blockers.
- Development Team: Developers and testers who create the product.

+ Advantages of Scrum:

- Faster problem-solving.
- Clear communication through daily standups.
- Continuous feedback and improvement.

2.2. Quality Management

Table 5. *Strategy for Meeting Quality Objectives*

Strategy	Expected Benefits
Defect prevention using standard defect-prevention guidelines and processes; apply coding standards developed in ABC.	10–20% reduction in defect injection rate and ~2% improvement in productivity.
Group review of program specifications for initial and logically complex use cases.	Improved quality through higher defect removal efficiency; early defect detection leading to productivity gains.
Group review of design documents and newly generated code by project leader, developer, and consultant.	Increased defect detection efficiency, reduced rework, and improved overall software quality.
Adoption of RUP (Rational Unified Process) methodology and implementation of the project in iterations, including milestone analysis and defect prevention exercises after each iteration.	~5% reduction in defect injection rate and ~1% improvement in overall productivity.

2.3. Test Strategy

Testing plan per sprint

- Sprint 1 – Requirement & Architecture (No functional code available), focus on:
    - Review of requirements and acceptance criteria.
    - Early static testing: SRS review, design review.
    - Prepare test environment, frameworks, naming conventions.
    - Create initial unit test structure.
- Sprint 2 – Development of Core Features:
    + Unit Test
        - Objective: Verify individual software modules as they are developed.
        - Technique: Write unit test cases using testing frameworks (e.g., Jest, Junit, Mocha, PHPUnit).
        - Completion Criteria:
            - Minimum X% unit test coverage for core modules.
            - No major defects.
    + Integration Tests:
        - Objective: Verify interactions between newly implemented modules (e.g., API ↔ Database).

        - Technique: Use stubs/mocks for unavailable components; test data flow and error handling.
        - Completion Criteria:
            - All integration scenarios defined for Sprint 2 features are executed.
    + Functional Tests:
        - Objective: Validate sprint user stories meet acceptance criteria.
        - Technique: Execute manual test cases from the product backlog.
        - Completion Criteria:
            - All Sprint 2 user stories "Done" according to Definition of Done (DoD).
- Sprint 3 – Fulltime-feature Integration & Enhancement:
    + Unit Tests:
        - Continue writing unit tests for new features.
        - Refactor or expand tests for changed modules.
    + Integration Tests:
        - Larger-scale integration (multi-module workflows).
        - Validate data consistency and API interactions.
    + Functional Tests:
        - Verify all features developed in Sprint 3.
    + Performance Tests (initial round):
        - Objective: Identify early performance issues.
        - Technique: Simulate moderate load and measure response times.
- Sprint 4 – Finalization & System Testing:
    + System Tests:
        - Objective: Validate the complete end-to-end system against requirements.
        - Technique: Full workflow testing with real or close-to-real data.
    + Performance Tests (final round):
        - Objective: Ensure system meets performance acceptance criteria.
        - Technique: Load testing, stress testing, spike testing using tools (e.g., Jmeter, k6).

+ Regression Tests:
    - Objective: Ensure new fixes do not break existing functions.
    - Technique: Run full regression suite.
+ User Acceptance Testing (UAT):
    - Gather feedback from end-users, stakeholders, or supervisor.

Test Environment and Tools:

+ Testing Framework: (e.g., Jest, Junit, Mocha, PHPUnit)
+ Development Environment: VS Code / Node.js / Spring Boot / etc.
+ Version Control: Git + GitHub
+ Bug Tracking: Jira / GitHub Issues
+ Performance Tools: Jmeter / k6

References:

+ Project Requirements Document
+ Architecture & Design Document
+ Coding Standards
+ Sprint Backlogs & Acceptance Criteria

2.4. Integration Testing Strategy

Integration Testing Plan per Sprint:

- Sprint 2 – Initial Integration (Core Modules)
    + Scope:
        - Begin integrating core modules such as backend API with database, or frontend components consuming basic API endpoints.
    + Activities:
        - Combine completed unit modules into functional pairs (e.g., authentication module ↔ database).
        - Use stubs and mock objects for components not yet completed.
        - Validate data flow, communication sequences, and basic error handling.
    + Completion Criteria:
        - All integration scenarios planned for Sprint 2 are executed.
        - No major integration defects remain open.
- Sprint 3 – Expanded Integration (Full Workflow)

+ Scope:
    - Integrate multiple modules into larger workflows (e.g., Login → Dashboard → CRUD features → Notifications).
    - Ensure cross-module consistency and correct data passing.
+ Activities:
    - Execute integration tests for complex workflows.
    - Validate dependency interactions, API chaining, session/state management, and error propagation.
    - Use mock servers or automated integration scripts where needed.
+ Completion Criteria:
    - All multi-module integration paths tested.
    - No critical issues affecting workflow execution.

- Sprint 4 – End-to-End Integration & System Validation
    + Scope:
        - Perform full-system integration testing across all components.
        - Validate the application behaves as a cohesive system without isolated module bugs.
    + Activities:
        - Execute end-to-end integration scenarios with production-like data.
        - Test all communication pathways, including API gateway, database operations, UI transitions, and external services (if any).
        - Run regression integration tests for all previously developed features.
    + Completion Criteria:
        - All integration test cases passed.
        - No high-severity integration defects.
        - System is ready for system testing and user acceptance testing.

Integration Testing Techniques

+ Combine units into subsystems and test interactions.
+ Use stubs and mock objects for incomplete or external modules.
+ Perform data flow testing, communication pathway testing, and error-handling validation.

+ Use automated scripts where possible to validate repeated module interactions.

Integration Testing Levels

+ Component Level: Interactions between two or more related modules.
+ Subsystem Level: Combined workflows across multiple components.
+ System Level: End-to-end interactions across the entire application.

Tools

+ Test Automation Frameworks: Jest, Junit, Mocha, PyTest
+ Mocking Frameworks: Mockito, Sinon.js, Jest Mocks
+ API Testing Tools: Postman, Newman
+ Virtualization Tools: Docker for environment consistency

Considerations

+ Test data: Use controlled data sets to ensure consistent test results.
+ Environment: Integration tests run in a stable, isolated test environment similar to production.
+ Coverage: All integration paths, both positive and negative, must be tested.
+ Defect tracking: All defects logged in Jira/GitHub Issues, prioritized and fixed before system testing.

2.5. System Testing Strategy

System Testing Execution Plan

- Sprint 4 — Full System Testing Phase
  + Scope:
    - The entire application, including all workflows, modules, UI/UX interactions, data flows, and external dependencies (if applicable).
  + Activities:
    - Execute end-to-end functional tests for all major workflows.
    - Validate all system-level requirements from the SRS.
    - Conduct non-functional testing, including performance, usability, and basic security checks.

    - Perform regression testing to ensure that recent fixes or final changes do not break existing features.
    - Test using realistic datasets to simulate real-world behavior.
    - Validate communication with any external APIs or services.

+ Completion Criteria:
    - All functional and non-functional requirements are verified.
    - No critical or high-severity defects remain open.
    - Performance meets defined acceptance thresholds.
    - System behaves correctly under typical and peak-load scenarios.

System Testing Techniques

- Functional Testing:
    + Verify that all system functionalities meet the Functional Requirements Specification (FRS/SRS).
    + Validate all user stories and acceptance criteria.
- Non-Functional Testing
    + Performance: response time, throughput, load behavior.
    + Security: role-based access, input validation, basic vulnerability checks.
    + Usability: UI behavior, ease of use, error messages, readability.
    + Compatibility: Browsers, screen sizes (if applicable).
- End-to-End Testing:
    + Execute full workflows that span multiple modules from start to finish.
    + Regression Testing
    + Run the full suite of executed tests after fixes or changes.
    + Ensure no new issues are introduced.

    Test Environment

    + System tests run in an environment as close as possible to production.
    + Includes complete backend, frontend, database, and external API connections.
    + Test data sets represent real-world datasets.

    Special Considerations

    + Complex real-world workflows must be tested carefully.

+ Tests involving multiple integrated components require accurate coordination.
+ External system dependencies may require mock servers or controlled test environments.
+ System testing must cover both positive flows and negative/error scenarios.

Completion Conditions

+ System testing is considered complete when:
+ All system-level acceptance tests pass.
+ All major workflows operate correctly end-to-end.
+ No critical/high defects remain unresolved.
+ The system meets both functional and non-functional acceptance criteria.
+ The supervisor/product owner accepts the system as ready for UAT or final demonstration.

3. Estimation

3.1. Effort

The Effort estimation is documented in the Product backlog.

3.2. Schedule

3.2.1. Project Milestone & Deliverables

Table 6. *Project Milestone & Deliverables*

No	Milestone	Time	Time Finish	Deliverables
1	Project Initialization	Aug 15th, 2025	Aug 20th, 2025	
2	Development			
2.1	Sprint 1	Aug 21th, 2025	Sep 10th, 2025	
2.2	Sprint 2	Sep 11th, 2025	Oct 15th, 2025	
2.3	Sprint 3	Oct 16th, 2025	Nov 20th, 2025	
2.4	Sprint 4	Nov 21th, 2025	Dec 17th, 2025	
3	Close Project			
3.1	Final meeting	Dec 17th, 2025	Dec 18th, 2025	
3.2	Final Presentation	Dec 20th, 2025	Dec 20th, 2025	

3.2.2. Work Breakdown Structure

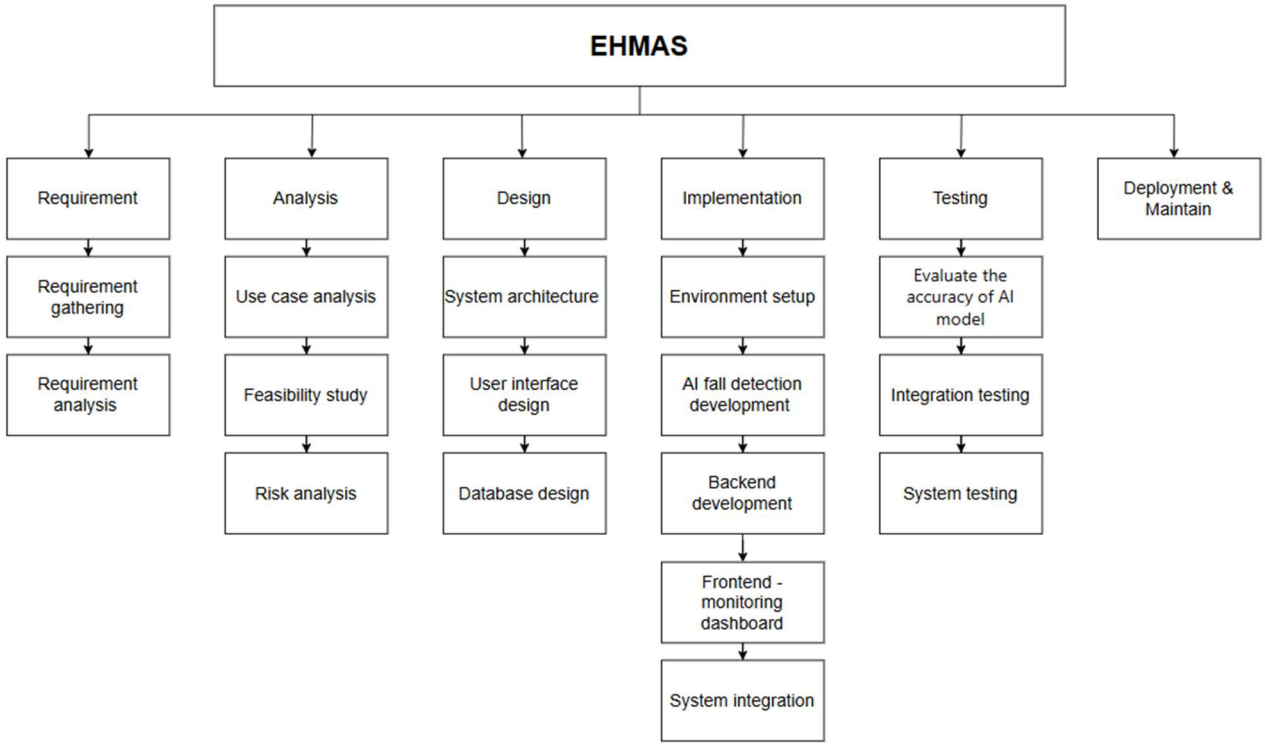

Figure 1. *Work breakdown structure*

3.2.3. Detailed Schedule

Table 7. Detailed schedule

No	Task Name	Effort (man-hours)	Start (Sprint)	Finish (Sprint)	Team Capacity
1.	Initialization	80	Aug 15th, 2025	Aug 20th, 2025	100
	Requirement Gathering	40			
	Proposal & Project Plan Docs	40			
2.	Development				
2.1	Sprint 1 (Environment Setup)	210	Aug 21th, 2025	Sep 10th, 2025	220
	Set up backend architecture (API, database connection)	100			
	Establish basic database	60			
	Implement user registration & login (JWT authentication)	50			
2.2	Sprint 2 (Core System Functions)	410	Sep 11th, 2025	Oct 15th, 2025	434
	Design basic web (register/login/profile)	60			
	Design API for Admin Device Management	60			
	Develop Fall Alert (video/image + timestamp)	60			
	Design basic web/app UI (Complete UI)	100			
	Data storage for fall events	60			
	Admin dashboard	70			
2.3	Sprint 3	460	Oct 16th, 2025	Nov 20th, 2025	500
	Integrate backend handle video or image	100			
	Connect backend with camera via Internet	80			
	Drink Water Reminder	70			
	Weight and BMI	70			
	Implement account & device approval	80			
	Optimize video/sensor data performance	60			
2.4	Sprint 4 (Testing & Refinement)	360	Nov 21th, 2025	Dec 17th, 2025	350
	Optimize performance	50			

	Code the interfaces of the remaining functions	40			
	Usability testing (friendly UI)	60			
	Performance & reliability testing	60			
	Debug and optimize UI/UX	60			
	Write User & Admin Manuals	60			
3	Close project	32			
	Final Project meeting	16	Dec 17th, 2025	Dec 18th, 2025	
	Final Release	16	Dec 20th, 2025	Dec 20th, 2025	

3.3. Resource

Table 8. *Resource*

Name	Roles	Responsibilities
Huynh Minh Thang	Scrum Master	Coordination, task management, AI integration
Vo Le Duy	Backend Developer	Server, database, alert system
Truong Manh Cuong	AI Developer, Backend Developer	Mobile/Web UI, caregiver dashboard
Nguyen Khac Nguyen Khoa	Frontend Developer	Testing, bug tracking, validation
Mai Van Dung	Backend Developer	AI dataset preparation, model training

3.4. Infrastructure

Table 9. *Infrastructure*

Work/Product	Purpose	Expected Availability	Note
Development Environment			
Visual studio code, PyCharm	Coding, creating the application	Initiation stage	Python for AI; React Native for app; Node.js/Express for backend
React	Operating System	Development stage	
Javascript	Development language for Web	Development stage	
MySQL	Store user data, fall events, caregiver info	Development stage	
TensorFlow, YOLO	Build and train AI fall detection model	Development stage	AI Frameworks

Hardware & Software			
Device: Laptop	Deployment Application		
Other Tools			
Git, Github	Source version control	Definition stage	
Trello	Project management tool	Initiation stage	
Zalo	Task tracking	Initiation stage	

3.5. Training Plan

Table 10. *Training plan*

Training Area	Participants	When, Duration	Waiver Criteria
Technical			
TensorFlow/PyTorch basics	All members	5 days	If already trained
React mobile app	All member	7 days	If already trained
Backend with Node.js	All member	5 days	If already trained
Python	All member	7 days	If already trained
Process			
Agile scrum	All members	1 day	If already trained
Data Security & Privacy	All members	2 days	
Group review	All members	1 day	Mandatory
Defect prevention	All members	1 day	Mandatory

3.6. Cost

3.6.1. Cost Person/Hours

Table 11. *Cost person/hours*

Full Name	Role	Salary Rate (USD/hour)
Huynh Minh Thang	Scrum Master	10
Vo Le Duy	Team Member	8
Truong Manh Cuong	Team Member	8
Nguyen Khac Nguyen Khoa	Team Member	8
Mai Van Dung	Team Member	8

3.6.2. Total Cost estimate

Table 12. *Total cost estimate*

No	Criteria	Price	Amount	Total (USD)
1	Working hours	$ 8.4	1880	$ 15,792
2	Party	$ 25	3	$ 75
3	Tools(camera)	$ 100	1	$ 100
4	Sensor	$ 50		$ 50
5	Other cost	$ 20		$ 20
Total cost				$ 16,037

Each sprint is 3 weeks long, with an average of 5 members working 4 hours/day.

- Total project duration: 17 Aug 2025 – 12 Dec 2025 (117 days ≈ 3.5 months)
- Team size: 5 members
- Average daily hours per member: 4 hours

Total person hours = 94 × 5 × 4 = 1,880 hours

Table 13. *Detailed cost description*

Description	Amount	Unit
Number of members	5	Person
Number of working hours per day	4	Hour
Number of workdays/weeks	6	Day
The duration of the project	3.5	Month
Party cost per member per time	8.4	USD
The number of working days	94	Day

4. Project Organization

The internal project organization ensures clear responsibilities, smooth collaboration, and effective communication. Since this project is developed by a student team, the organization emphasizes both technical development and learning outcomes, with guidance from the mentor.

The project is dependent on the collaborative effort of all team members, proper communication, and external factors such as university deadlines, access to datasets, and required software/hardware for AI training and testing.

4.1. Organization Structure

The project adopts the Scrum framework, dividing work into short iterations (sprints) and assigning responsibilities according to Scrum roles. This approach supports flexibility, iterative progress, and active team involvement.

Subprojects / Areas of Responsibility:

- AI & Data Processing: Data collection, preprocessing, training, and optimization of fall detection models.
- Backend Development: API services, database management, and alert notification system.
- Frontend Development: User-friendly mobile/web application for caregivers and elders.
- Quality Assurance: Testing AI accuracy, app usability, performance, and data security.
- Deployment/Integration: Setting up cloud services and ensuring reliable alert delivery.
- Steering Functions: Managed by the Scrum Master, who facilitates Scrum and resolves blockers.
- Project Management Functions: Mentor as Product Owner, overseeing deliverables and alignment with objectives.
- Execution Functions: All team members contribute to coding, testing, documentation, and reporting.

4.2. Project Team

- Scrum Master: Responsible for facilitating Scrum processes and ensuring the team follows agile practices.
- Project Manager/Product Owner: Responsible for overall project oversight, coordinating activities, and ensuring project alignment with goals.
- Frontend Developer(s): Responsible for developing the mobile/web application interface for caregivers and elders, ensuring usability and accessibility.
- Backend Developer(s): Handles server-side logic, database management, and the alert notification system.
- AI & Data Engineer(s): Collects and preprocesses data, trains and optimizes the AI model for fall detection, and evaluates accuracy.

- Quality Assurance Team: Conducts system testing (functionality, reliability, security), validates AI performance, and ensures the system meets quality standards.

4.2.1. Scrum Team & Responsibilities

Table 14. *Scrum Team*

ID	Name	Role	Contact Email
28219201080	Thang, Huynh Minh	Scrum Master	thang05112004@gmail.com
27211202824	Duy, Vo Le	Backend Developer	duyvole9@gmail.com
28211149985	Khoa, Nguyen Khac Nguyen	Frontend Developer	nguyenkhoa14042004@gmail.com
28211247724	Cuong, Truong Manh	AI Developer, Backend Developer	tmanhcuong2k4@gmai.com
28219025364	Dung, Mai Van	Backend Developer	maivandung277@gmail.com

Table 15. *Responsibilities*

Role	Responsibility	Name/Title
Product Owner	- Understand the user and customers with their needs. - Collaborate with the development team. - Manage the stakeholders. - Describe the user experience and product features. - Provides detail user stories.	Jan Samuelsson
Scrum Master	- Communicate the value of Scrum - Teach the organization on Scrum to maximize business value - Attend all Scrum meetings - Preserve the integrity and spirit of the Scrum framework - Maintain the focus of the Team - Make the Team aware of impediments and facilitate efforts to resolve them - Serve as a coach and mentor to members of the Team - Respectfully hold the Team, Product Owner and Stakeholders accountable for their commitments - Continually work with the Team and business to find and implement improvements	Huynh Minh Thang

Secretary	- Record the content of group meetings and activities of the member	All
Reviewer	- Review documents	All
Developer	- Analysis of the functions and requirements of the product. - Code and test. - Fix error.	All
Analyzer	- Gather user stories. - Analysis user story to specify Document.	All
Tester	- Do the Test plan - Creation of test designs, test processes, test cases and test data. - Carry out testing as per the defined procedures. - Graph the results and make sure people know when test results decline. - Prepare all reports related to software testing carried out. - Analysis and evaluate the Test result. - Ensure that all tested related work is carried out as per the defined standards and procedures.	All
Mentor	- Guide on the process. - Monitoring all activities of Team. - Help with anything.	Jan Samuelsson

5. Communication & Reporting

5.1. Communication methodology

Table 16. *Communication methodology*

Audience/ Attendees	Topic/ Deliverable	Frequency	Method
Mentor and Team member	Project Progress Review	Every 2 weeks	in-person meeting, Email, Zalo
Team Member	Project Progress Review and Daily Meeting	Daily	Zalo, Zoom

5.2. Communication And Report

Table 17. *Communication and report*

Type of communication	Methods, tools	Frequency	Information	People
Communication among in group				
Scrum meeting	Zoom, Zalo, Discord	Every two days	Informed about what was done in the last 24 hours, working on plans for today, the difficulties encountered and the solutions required, just meeting 10-15 minutes.	Project team
Sprint Planning Meeting	Meet face to face	Every sprint in the beginning	All members in team together to analyze the requirements, functions, working on the sprint going to do, planning and design for the sprint.	Project team
Retrospective meeting	Meet face to face	after sprint	Complete documentation. For each stage, sharing materials, given the strengths and weaknesses for each. Period for each member and the solution calculated measurement project.	Project team and Mentor

6. Configuration Management

<Refer to the CM plan or insert here the contents of the CM plan as appropriated>

7. References

- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... & Thomas, D. (2001). Manifesto for Agile Software Development.
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. Scrum.org.
- Google Developers. (n.d.). React – A JavaScript library for building user interfaces.

Definitions And References

Acronym	Definition	Note
PM	Project Manager	
QA	Quality Assurance	
TP	Test Plan	
TC	Test Case	
SC	Source Code	
CM	Configuration Management	